

High-Level Design (HLD)

RAG-Based Customer Support Assistant

1. Project Objective

Design and implement a Retrieval-Augmented Generation system that processes a customer support PDF knowledge base, retrieves relevant information using embeddings, answers user queries contextually, uses graph-based workflow control through LangGraph, routes responses based on intent, and supports Human-in-the-Loop escalation.

2. System Overview

The system acts as a customer support assistant. It reads customer support Q&A content from a PDF, stores the information as vector embeddings in ChromaDB, retrieves relevant chunks for customer questions, and returns grounded answers. If the query is sensitive, vague, or low confidence, the workflow escalates it to a human reviewer.

Problem Definition

Customer support teams repeatedly answer common questions related to account management, billing, orders, delivery, technical issues, refunds, and returns. Manual search through documents is slow and inconsistent. This RAG assistant improves response speed and consistency by using the support knowledge base as the source of truth.

Scope of the System

- PDF-based knowledge base ingestion
- Chunk creation from extracted document text
- Embedding generation for chunks and user queries
- Vector storage and retrieval using ChromaDB
- LangGraph workflow orchestration
- Intent routing for customer support categories
- Human-in-the-Loop escalation for risky or uncertain requests

3. Architecture Diagram

Architecture flow:

1. Customer asks a question through the CLI.
2. LangGraph receives the input state.
3. The processing node detects intent and retrieves relevant chunks.
4. ChromaDB returns the nearest matching document chunks.
5. HITL logic checks confidence and escalation rules.
6. The system either generates an answer or escalates to a human reviewer.
7. The final output is displayed to the user.

Architecture Components

Component	Purpose
User Interface	Command-line interface through main.py for ingestion, direct ask, and interactive chat.
Document Loader	Reads customer_support_q&a.pdf and extracts page-level text.
Chunking System	Splits extracted text into overlapping chunks for retrieval.
Embedding System	Creates deterministic local vectors for chunks and queries.
Vector Database	Stores embeddings and metadata in ChromaDB.
Retrieval Layer	Finds the most relevant support chunks for a user query.
LLM/Answer Layer	Generates a final answer from retrieved context.
LangGraph Workflow	Controls the process, human review, and output nodes.
HITL Module	Escalates sensitive, vague, missing-context, or low-confidence questions.

Component Interaction Explanation:

- The User Interface (CLI/Web) captures the user query and sends it to the query processing pipeline.
- The Router (Intent Detection Layer) analyzes the query and determines the appropriate category (e.g., billing, orders, technical).
- The Retriever interacts with the Vector Database (ChromaDB) to fetch top-k relevant chunks using embeddings.
- The Embedding System ensures both documents and queries are converted into the same vector space.
- The LLM Processing Layer uses retrieved context to generate grounded responses.
- The Workflow Orchestration (LangGraph) controls execution flow between nodes.
- The HITL System evaluates confidence and decides whether to escalate to a human.

4. Data Flow

1. PDF document is ingested and split into chunks
2. Each chunk is converted into embeddings
3. Embeddings are stored in ChromaDB
4. User submits a query
5. Query is embedded using the same embedding model
6. Retriever fetches top-k similar chunks
7. Router identifies query intent

8. HITL checks confidence and safety
9. LLM generates final response OR escalates

5. Technology Choices

Technology	Reason
Python	Simple and widely used for AI/RAG prototyping.
PyPDF	Extracts text from the PDF knowledge base.
ChromaDB	Lightweight persistent vector database suitable for local projects.
LangGraph	Provides explicit graph-based workflow and conditional routing.
Local Embeddings	Keeps the project runnable without external API keys.
OpenAI Package	Optional upgrade path for richer answer generation.

Why ChromaDB:

- Lightweight, easy local setup
- Efficient similarity search for embeddings
- No dependency on external infrastructure

Why LangGraph:

- Enables explicit workflow control
- Supports conditional routing and state transitions
- Makes debugging and scaling easier

LLM Choice:

- Selected based on availability for local/offline usage
- Prioritizes deterministic and reproducible outputs

6. Scalability Considerations

1. For larger knowledge bases, ingestion can be batched by document.
2. ChromaDB can be replaced by a managed vector database in production.
3. Frequently asked questions can be cached to reduce latency.
4. The CLI can be extended into a web interface or API.
5. HITL review can integrate with ticketing tools or supervisor dashboards.
6. Chunking with overlap ensures scalability without losing context

7. Vector search scales efficiently; can be optimized using indexing
8. Reduce top-k, optimize chunk size, and cache embeddings